

와인 데이터를 활용한 와인 품질 예측

- 도메인 : 품질 관리(QC)
 - 목 표 : 와인의 품질을 예측하는 인공지능모델 개발
 - 와인 도매업체인 C사에서 수입하는 와인 품질이 경쟁사 와인 품질 대비 떨어진다는 평가를 받으며 매출이 감소하고 있다.
 - 계속되는 매출 부진을 만회하기 위해, C사 품질관리팀장은 수십년간 축적된 공신력 있는 와인평론가의 와인 평가 데이터를 활용해서 '와인품질등급'을 예측하는 인공지능 모델을 도입하려고 한다.
 - C사는 브랜드파워가 있기 때문에, 인공지능을 활용해서 '와인품질등급'의 정확도가 개선되어 고품질의 와인만 선별하여 수입하면 매출이 빠르게 회복될 것으로 기대하고 있다.
-

[유의사항]

- 각 문항의 답안은 반드시 '#여기에 답안코드를 작성하고 실행하세요', '#여기에 답안을 입력하세요' 등이 표시된 셀(cell)에 입력해야 합니다
 - 제공된 시험문항 셀을 삭제하거나 답안 위치가 아닌 다른 셀에 답안코드를 작성 시 채점되지 않습니다
 - 답안 작성 전에 문항에 제시된 가이드를 확인하세요
 - 문항에 변수명이 제시된 경우 반드시 해당 변수명을 사용하세요
 - 시험 중에는 상단의 '임시저장' 버튼을 수시로 클릭해 저장해주시고, 답안 제출시에는 '최종제출' 버튼을 클릭해주시기 바랍니다
 - 시험 문항과 데이터 등을 무단으로 촬영/캡처, 복사/복제, 판매/재판매, 공개/전재, 공유/유포 할 경우 지식재산권 침해에 따른 법적 제재를 받을 수 있습니다
 - 오픈북은 허용된 사이트만 참고 가능하며 그 외 자료, 사이트 등을 참고하는 경우 부정행위로 간주됩니다
-

[데이터 컬럼 설명 (데이터 파일명: wine_quality_data.csv)]

- Grade : 와인품질등급(Pass, Fail)
- E_title : 와인이름
- E_country : 생산국가
- E_province : 생산지 세부주소 ①
- E_region_1 : 생산지 세부주소 ②
- E_region_2 : 생산지 세부주소 ③
- E_winery : 와이너리(와인 생산 농장명)
- E_FE_vintage : 포도수확연도
- E_wine_variety : 포도품종
- E_FE_wine_kind : 와인종류
- FE_continent : 생산대륙(신대륙/구대륙)

- E_price : 가격
- FE_points_location : 와인 생산지 평점
- FE_points_variety : 포도 품종 평점
- FE_points_winery. : 와이너리 평점

<데이터 분석 (20점)>

1. scikit-learn은 머신러닝에 널리 사용되는 파이썬 라이브러리입니다.

scikit-learn을 사용할 수 있게 별칭(alias)을 sk로 해서 불러오세요.

In [1]: *# (1) 여기에 답안코드를 작성하고 실행하세요.*

```
import sklearn as sk
```

다음 문항을 풀기 전에 아래 코드를 실행하세요.

In [2]:

```
import numpy as np
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

In [3]:

```
# 리눅스
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
plt.rc('font', family='NanumGothicCoding')

# 윈도우용으로 아래 한글 나오게 하기 위해 사용함.
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib as mpl
import matplotlib.font_manager as fm
font_path = 'C:\\Windows\\Fonts\\gulim.ttc'
```

```
font = fm.FontProperties(fname=font_path).get_name()
mpl.rc('font', family=font)

import warnings
warnings.filterwarnings('ignore')
```

2. 시 모델링을 위해 분석 및 처리할 데이터 파일을 읽어오려고 합니다.

pandas로 데이터 파일을 읽어온 뒤, 데이터프레임 변수명 df에 할당하고 첫 4개 행을 출력하세요.

- 데이터프레임 변수명 : df
- 데이터 파일명 : wine_quality_data.csv
 - csv 파일은 본 문제/답안지와 동일한 경로에 있습니다

In [4]: # (2) 여기에 답안코드를 작성하고 실행하세요

```
data = pd.read_csv('./wine_quality_data.csv')
data.head(4)
```

Out[4]:

	Grade	E_title	E_country	E_province	E_region_1	E_region_2	E_winery	E_FE_v
0	Fail	Nicosia 2013 Vulkà Bianco (Etna)	Italy	Sicily & Sardinia	Etna	-	Nicosia	
1	Fail	Quinta dos Avidagos 2011 Avidagos Red (Douro)	Portugal	Douro	-	-	Quinta dos Avidagos	
2	Fail	St. Julian 2013 Reserve Late Harvest Riesling ...	US	Michigan	Lake Michigan Shore	-	St. Julian	
3	Fail	Tandem 2011 Ars In Vitro Tempranillo- Merlot (N...	Spain	Northern Spain	Navarra	-	Tandem	

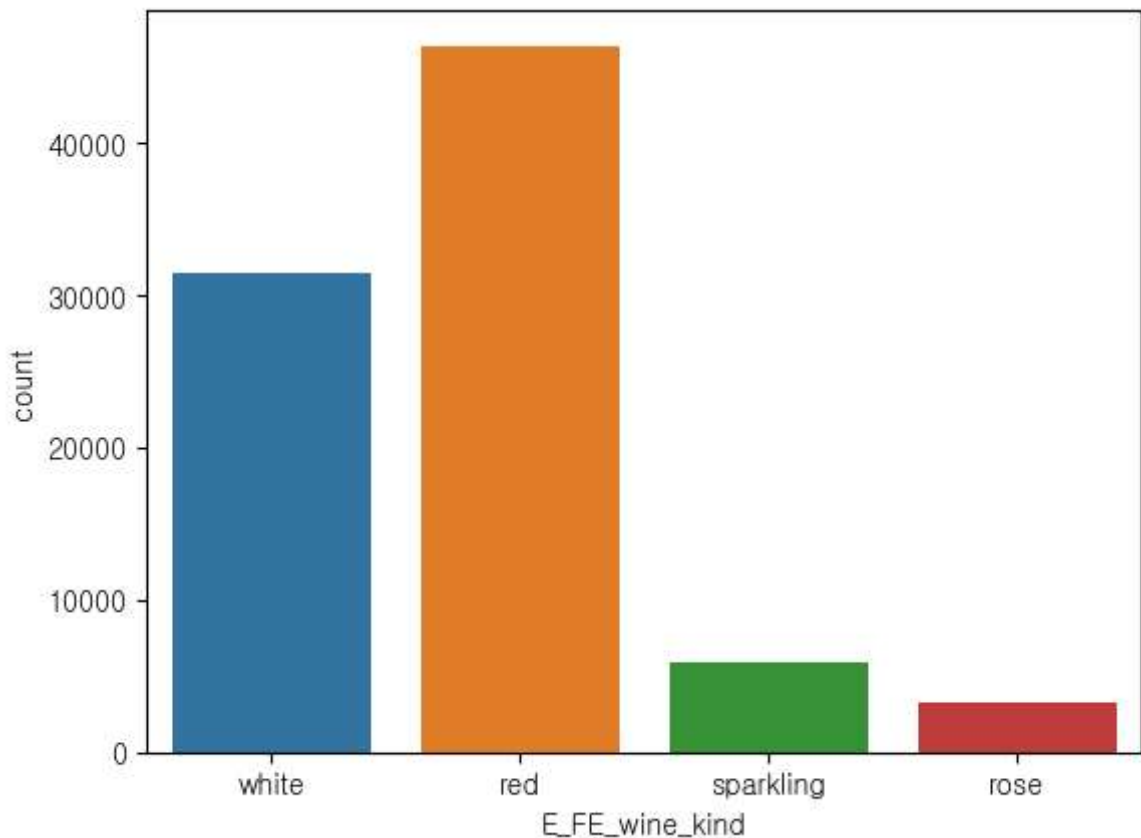
3. 시각화를 통해 와인종류(E_FE_wine_kind)의 분포를 파악하려 합니다.

- (3-1) 아래 가이드에 따라 seaborn 라이브러리를 활용해서 시각화하세요
 - 대상 데이터프레임 : data
 - 와인종류의 분포를 보여주는 countplot 그래프를 그리세요
- (3-2) 출력된 그래프에서 와인 평론가들이 가장 많이 평가한 와인종류는 무엇인가요?

In [5]: # (3-1) 여기에 답안코드를 작성하고 실행하세요

```
sns.countplot(data=data, x='E_FE_wine_kind')
```

Out[5]: <Axes: xlabel='E_FE_wine_kind', ylabel='count'>



In []: # (3-2) 여기에 답안을 입력하세요(실행 불필요)

red

4. 그래프를 통해 와인품질등급(Grade) 별 와이너리 평점(FE_points_winery) 컬럼의 분포를 확인하려 합니다.

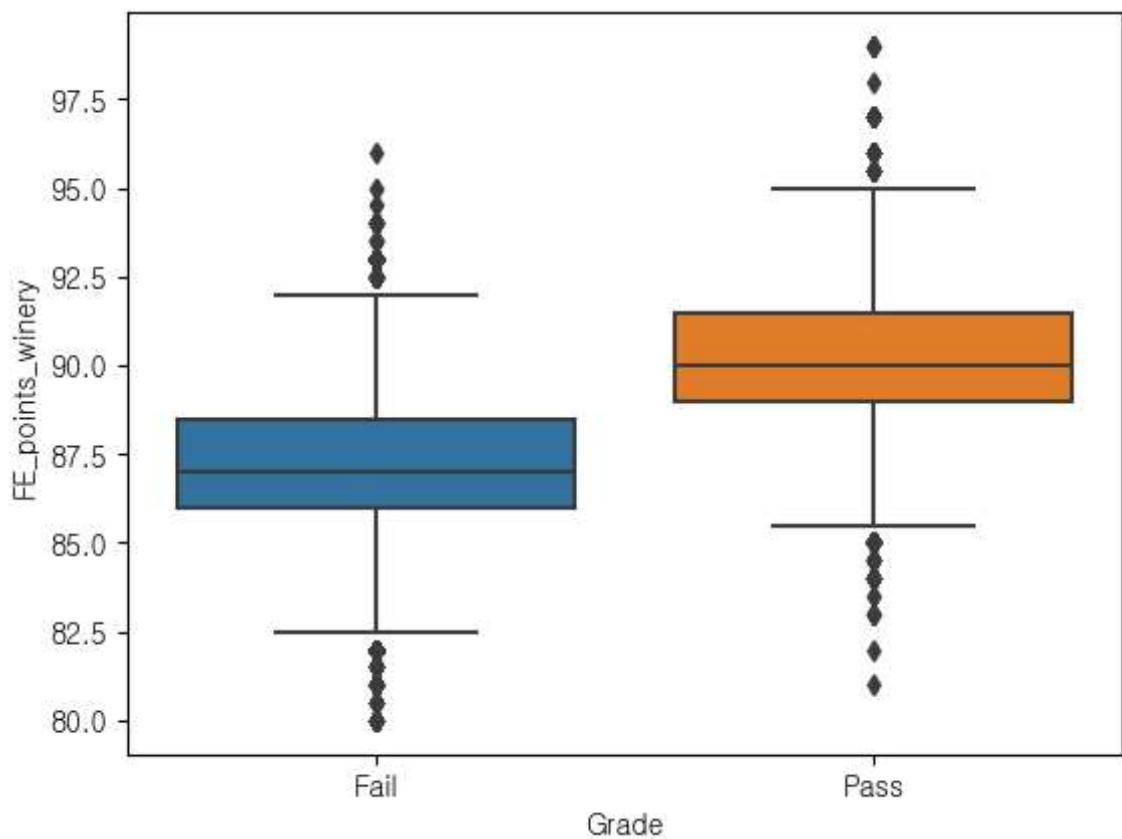
아래 가이드에 따라 boxplot으로 시각화하세요.

- 대상 데이터프레임 : data
- seaborn의 boxplot으로 와이너리 평점(FE_points_winery) 컬럼의 분포를 그래프로 그리세요
- X축에는 와인품질등급(Grade)을, Y축에는 와이너리 평점(FE_points_winery)을 표시하세요

In [6]: # (4) 여기에 답안코드를 작성하고 실행하세요

```
sns.boxplot(data=data, x='Grade', y='FE_points_winery')
```

Out[6]: <Axes: xlabel='Grade', ylabel='FE_points_winery'>



<데이터 전처리 (30점)>

5. boxplot으로 확인한 결과, 와이너리 평점 (FE_points winery) 컬럼에서 이상치가 발견되었습니다. AI 모델링할때 부정적 영향이 없도록 아래 가이드에 따라 이상치를 처리하세요.

- 대상 데이터프레임 : data
- 와이너리 평점이 upperfence 값보다 큰 이상치가 있는 행(row)을 삭제하세요
- 와이너리 평점이 lowerfence 값보다 작은 이상치가 있는 행(row)을 삭제하세요
- AI모델링에 불필요한 와인이름(E_title) 컬럼을 삭제하세요
- 전처리를 적용한 데이터프레임은 data_temp라는 변수명으로 저장하세요
- 아래 코드 셀(cell)에 있는 코드의 빈칸을 채우고 실행하세요
- 빈칸에 들어갈 내용은 각 답안 셀(cell)에 입력하세요 (#5-1, #5-2)

In []: # (코드 셀) 코드의 빈칸을 채우고 실행하세요

```
q1 = data['FE_points_winery'].quantile(0.25)
q3 = data['FE_points_winery'].quantile(0.75)
iqr = q3 - q1

lower_fence = q1 - 1.5 * iqr
upper_fence = q3 + 1.5 * iqr

data_temp = data. <#5-1> ( data[(data['FE_points_winery'] > upper_fence) | (data
data_temp = data_temp.drop( columns=['E_title'], axis=1)
```

In []: # (5-1) 여기에 답안을 입력하세요(실행 불필요)

drop

In []: # (5-2) 여기에 답안을 입력하세요(실행 불필요)

index

In [7]: # [정답]

```
q1 = data['FE_points_winery'].quantile(0.25)
q3 = data['FE_points_winery'].quantile(0.75)
iqr = q3 - q1

lower_fence = q1 - 1.5 * iqr
upper_fence = q3 + 1.5 * iqr

data_temp = data.drop( data[(data['FE_points_winery'] > upper_fence) | (data['FE
data_temp = data_temp.drop( columns=['E_title'], axis=1)
data_temp.reset_index(drop=True, inplace=True)
```

6. 데이터에 결측치가 있을 경우 AI모델링의 성능이 저하될 수 있습니다. AI모델링할때 부정적 영향이 없도록 아래 가이

드에 따라 결측치를 처리하세요.

- 대상 데이터프레임 : data_temp
 - 결측치 개수를 확인한 후에 결측치가 있는 행을 모두 삭제하세요
 - 전처리가 반영된 데이터프레임은 변수명 data_na에 저장하세요
 - 결측치가 사라졌는지 확인하세요
 - 아래 코드 셀(cell)에 있는 코드의 빈칸을 채우고 실행하세요
 - 빈칸에 들어갈 내용은 각 답안 셀(cell)에 입력하세요 (#6-1, #6-2)
-

```
In [ ]: # (코드 셀) 코드의 빈칸을 채우고 실행하세요

print( '결측치 처리 전\n', data_temp. <#6-1> ().sum() )
data_na = data_temp. <#6-2> ()
print( '결측치 처리 후\n', data_na. <#6-1> ().sum() )
```

```
In [ ]: # (6-1) 여기에 답안을 입력하세요(실행 불필요)

isnull
```

```
In [ ]: # (6-2) 여기에 답안을 입력하세요(실행 불필요)

dropna
```

```
In [8]: # [정답]

print( '결측치 처리 전\n', data_temp.isnull().sum() )
data_na = data_temp.dropna()
print( '결측치 처리 후\n', data_na.isnull().sum() )
```

```

결측치 처리 전
  Grade      0
E_country    0
E_province    0
E_region_1    0
E_region_2    0
E_winery      0
E_FE_vintage  0
E_wine_variety 0
E_FE_wine_kind 3746
FE_continent  0
E_price      6902
FE_points_location 0
FE_points_variety 0
FE_points_winery 0
dtype: int64
결측치 처리 후
  Grade      0
E_country    0
E_province    0
E_region_1    0
E_region_2    0
E_winery      0
E_FE_vintage  0
E_wine_variety 0
E_FE_wine_kind 0
FE_continent  0
E_price      0
FE_points_location 0
FE_points_variety 0
FE_points_winery 0
dtype: int64

```

7. 원-핫 인코딩(One-Hot Encoding)은 범주형 데이터를 1과 0의 이진형 벡터로 변환하기 위해 사용하는 방법입니다.

아래 가이드에 따라 범주형 데이터를 이진형 벡터로 변환하는 코드를 완성하세요.

-
- (7-1) 아래 가이드에 따라 결측치를 처리하고 검증하는데 에러가 나고 있습니다. 코드의 오류를 정정하세요
 - 대상 데이터프레임 : data_na
 - 원-핫 인코딩 대상 : 'E_country', 'E_FE_wine_kind'
 - 원-핫인코딩이 반영된 결과를 데이터프레임 변수명 data_preset에 저장하세요
 - 원-핫인코딩한 결과, 인코딩 대상 컬럼이 변환되었는지 info 함수로 검증하세요
 - (7-2) data_preset의 전체 컬럼 개수는 총 몇 개인가요?
-

```

In [ ]: # (7-1) 여기에 코드의 오류를 정정하고 실행하세요

data_na = data_na.drop( ['E_province', 'E_region_1', 'E_region_2', 'E_winery', '

```

```
data_preset = pd.one_hot_encoding(data = data_na, columns = ['E_country', 'E_FE_
data_preset.info()
```

In []: *# (7-2) 여기에 답안을 입력하세요(실행 불필요)*

50

In [9]: *# [정답]*

```
data_na = data_na.drop( ['E_province', 'E_region_1', 'E_region_2', 'E_winery', '
data_preset = pd.get_dummies(data = data_na, columns = ['E_country', 'E_FE_wine_
data_preset.info()
```

<class 'pandas.core.frame.DataFrame'>

Index: 79924 entries, 2 to 90059

Data columns (total 50 columns):

#	Column	Non-Null Count	Dtype
0	Grade	79924 non-null	object
1	E_FE_vintage	79924 non-null	int64
2	FE_continent	79924 non-null	int64
3	E_price	79924 non-null	float64
4	FE_points_location	79924 non-null	float64
5	FE_points_variety	79924 non-null	float64
6	FE_points_winery	79924 non-null	float64
7	E_country_Armenia	79924 non-null	bool
8	E_country_Australia	79924 non-null	bool
9	E_country_Austria	79924 non-null	bool
10	E_country_Bosnia and Herzegovina	79924 non-null	bool
11	E_country_Brazil	79924 non-null	bool
12	E_country_Bulgaria	79924 non-null	bool
13	E_country_Canada	79924 non-null	bool
14	E_country_Chile	79924 non-null	bool
15	E_country_Croatia	79924 non-null	bool
16	E_country_Cyprus	79924 non-null	bool
17	E_country_Czech Republic	79924 non-null	bool
18	E_country_England	79924 non-null	bool
19	E_country_France	79924 non-null	bool
20	E_country_Georgia	79924 non-null	bool
21	E_country_Germany	79924 non-null	bool
22	E_country_Greece	79924 non-null	bool
23	E_country_Hungary	79924 non-null	bool
24	E_country_India	79924 non-null	bool
25	E_country_Israel	79924 non-null	bool
26	E_country_Italy	79924 non-null	bool
27	E_country_Lebanon	79924 non-null	bool
28	E_country_Luxembourg	79924 non-null	bool
29	E_country_Macedonia	79924 non-null	bool
30	E_country_Mexico	79924 non-null	bool
31	E_country_Moldova	79924 non-null	bool
32	E_country_Morocco	79924 non-null	bool
33	E_country_New Zealand	79924 non-null	bool
34	E_country_Peru	79924 non-null	bool
35	E_country_Portugal	79924 non-null	bool
36	E_country_Romania	79924 non-null	bool
37	E_country_Serbia	79924 non-null	bool
38	E_country_Slovakia	79924 non-null	bool
39	E_country_Slovenia	79924 non-null	bool
40	E_country_South Africa	79924 non-null	bool
41	E_country_Spain	79924 non-null	bool
42	E_country_Switzerland	79924 non-null	bool
43	E_country_Turkey	79924 non-null	bool
44	E_country_US	79924 non-null	bool
45	E_country_Ukraine	79924 non-null	bool
46	E_country_Uruguay	79924 non-null	bool
47	E_FE_wine_kind_rose	79924 non-null	bool
48	E_FE_wine_kind_sparkling	79924 non-null	bool
49	E_FE_wine_kind_white	79924 non-null	bool

dtypes: bool(43), float64(4), int64(2), object(1)

memory usage: 8.2+ MB

8. 모델링후에 모델의 성능을 평가할 수 있도록 훈련과 검증 각각에 사용할 데이터셋을 분리하려고 합니다.

와인품질등급(Grade) 컬럼을 레이블값 y로, 나머지 컬럼을 피쳐값 X로 할당한 후 훈련데이터셋과 검증데이터셋으로 분리하세요.

-
- 대상 데이터프레임 : data_preset
 - 훈련과 검증데이터셋 분리
 - 훈련데이터셋 레이블 : y_train, 훈련데이터셋 피쳐: X_train
 - 검증데이터셋 레이블 : y_valid, 검증데이터셋 피쳐: X_valid
 - 훈련데이터셋과 검증데이터셋 비율은 70:30으로 설정하세요
 - 훈련데이터셋과 검증데이터셋의 와인등급 비율을 동일하게 하는 옵션을 설정하세요.
 - 난수 시드는 7로 설정하세요
 - scikit-learn의 train_test_split 함수를 활용하세요
-

In [10]: *# (8) 여기에 답안코드를 작성하고 실행하세요*

```
from sklearn.model_selection import train_test_split

X = data_preset.drop('Grade', axis=1)
y = data_preset['Grade']

X_train, X_valid, y_train, y_valid = train_test_split(X, y, test_size=0.3, randc
```

9. 스케일링을 통해 데이터 이상치의 영향도를 낮추려고 합니다.

- (9-1) 아래 가이드에 따라 스케일링 처리하는데 에러가 나고 있습니다. 코드의 오류를 정정하세요
 - sklearn.preprocessing에서 StandardScaler 함수를 import하세요
 - StandardScaler 함수를 ss 변수에 지정하세요
 - 훈련데이터에 대해 StandardScaler를 학습시키고 변환을 적용하세요
 - 검증데이터에 학습된 스케일링 기준을 그대로 적용하세요
 - 스케일링된 검증데이터의 가장 큰 값을 찾고, 이를 반올림하세요
- (9-2) StandardScaler 적용 후 검증데이터의 최댓값을 반올림한 결과는 무엇입니까?

```
In [ ]: # (9-1) 여기에 코드의 오류를 정정하고 실행하세요

from sklearn.preprocessing import StandardScaler

ss = StandardScaler()
X_train = ss.fit_transform(X_train)
X_valid = ss.fit_transform(X_valid)
round(np.max(X_valid))
```

```
In [ ]: # (9-2) 여기에 답안을 입력하세요(실행 불필요)

237
```

```
In [11]: # [정답]

from sklearn.preprocessing import StandardScaler

ss = StandardScaler()
X_train = ss.fit_transform(X_train)
X_valid = ss.transform(X_valid)
round(np.max(X_valid))
```

Out[11]: 237

<AI 모델링(50점)>

10. 와인품질등급(Grade)을 예측하는 머신러닝 모델을 만들려고 합니다.

DecisionTree와 RandomForest는 여러 규칙을 순차적으로 적용하면서 독립변수 공간을 분할하는 모형으로서, 분류와 회귀 분석에 모두 사용될 수 있습니다.

- (10-1) DecisionTree 모델의 하이퍼파라미터를 아래와 같이 구성하고, 변수에 저장하세요
 - 트리의 최대 깊이 : 5
 - 노드를 분할하기 위한 최소한의 샘플 데이터수(min_samples_split) : 3
 - 난수 시드 : 7
 - Decision Tree 모델을 dt 변수에 저장하세요
 - 앞서 분리한 훈련데이터셋으로 fit 함수를 사용해 학습시키세요
- (10-2) RandomForest 모델의 하이퍼파라미터를 아래와 같이 설정하고, 변수에 저장하세요
 - 트리의 최대 깊이 : 5
 - 노드를 분할하기 위한 최소한의 샘플 데이터수(min_samples_split) : 3
 - 난수 시드 : 7
 - Random Forest 모델을 rf 변수에 저장하세요
 - 앞서 분리한 훈련데이터셋으로 fit 함수를 사용해 학습시키세요

In [12]: # (10-1) 여기에 답안코드를 작성하고 실행하세요

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(max_depth=5, min_samples_split=3, random_state=7)
dt.fit(X_train, y_train)
```

Out[12]: ▼ DecisionTreeClassifier

DecisionTreeClassifier(max_depth=5, min_samples_split=3, random_state=7)

In [13]: # (10-2) 여기에 답안코드를 작성하고 실행하세요

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(max_depth=5, min_samples_split=3, random_state=7)
rf.fit(X_train, y_train)
```

Out[13]: ▼ RandomForestClassifier

RandomForestClassifier(max_depth=5, min_samples_split=3, random_state=7)

11. 모델의 해석력과 개선 방향을 이해하기 위해 RandomForest 모델의 변수중요도를 파악하려고 합니다.

- (11-1) 아래 가이드에 따라 모델의 변수중요도 파악하는데 에러가 나고 있습니다.
코드의 오류를 수정하세요
 - 변수중요도 점수와 변수 이름을 매핑하여 DataFrame을 만드세요
 - 변수중요도 점수를 기준으로 내림차순 정렬하고 top 5개만 선정하세요.
 - 변수중요도 점수를 막대 그래프로 시각화하세요
- (11-2) 출력된 그래프에서 가장 높은 중요도를 보이는 변수명을 쓰세요

In []: # (11-1) 여기에 코드의 오류를 정정하고 실행하세요

```
fi = pd.DataFrame({'feature': X.columns, 'importance': rf.importances})  
fi = fi.sort_index('importance', ascending=False)[:10]  
sns.barplot(x='importance', y='feature', data=fi, palette='viridis')
```

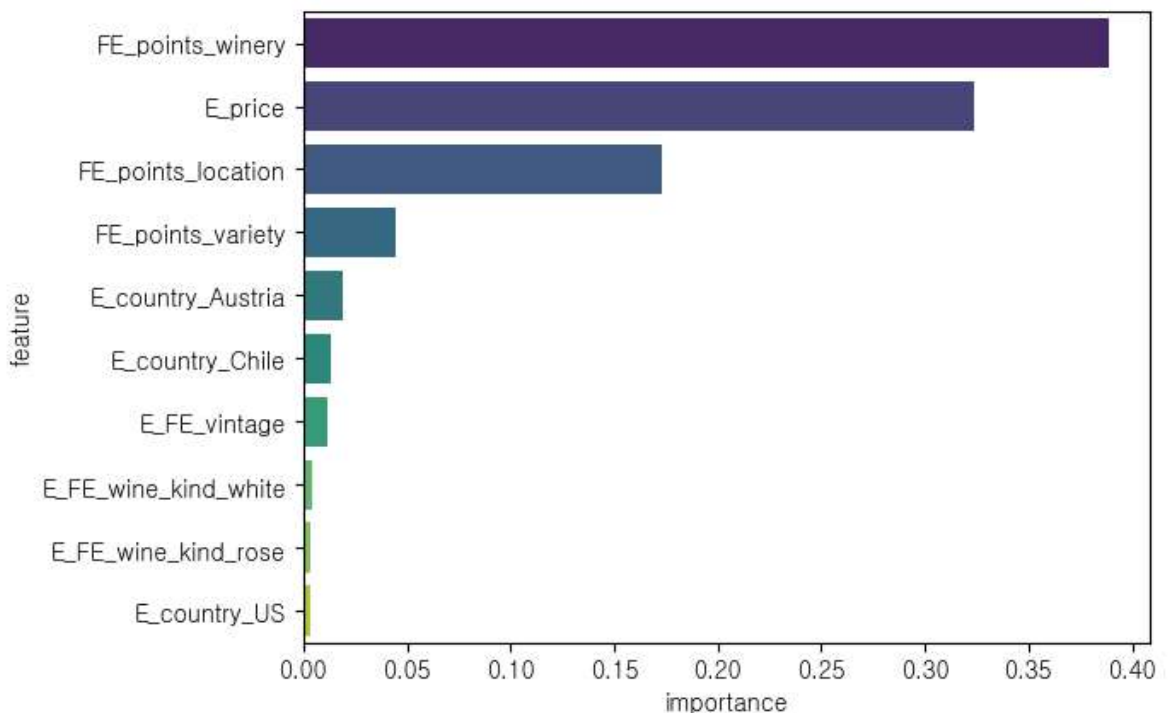
In []: # (11-2) 여기에 답안을 입력하세요(실행 불필요)

FE_points_winery

In [14]: # [정답]

```
fi = pd.DataFrame({'feature': X.columns, 'importance': rf.feature_importances_})  
fi = fi.sort_values('importance', ascending=False)[:10]  
sns.barplot(x='importance', y='feature', data=fi, palette='viridis')
```

Out[14]: <Axes: xlabel='importance', ylabel='feature'>



12. 앞서 학습시킨 DecisionTree와 RandomForest 모델의 성능을 평가하려고 합니다.

- (12-1) 아래 가이드에 따라 각 모델의 Accuracy를 산출하려는데 에러가 나고 있습니다. 코드의 오류를 정정하세요
 - 성능 평가는 검증데이터셋을 활용하세요
 - 앞서 만든 DecisionTree 모델로 y값을 예측하고, 그 결과를 y_pred_dt 변수에 저장하세요
 - 검증 정답(y_valid)과 예측값(y_pred_dt)의 Accuracy를 구하고 dt_acc 변수에 저장하세요
 - 앞서 만든 RandomForest 모델로 y값을 예측하고, 그 결과를 y_pred_rf에 저장하세요
 - 검증 정답(y_valid)과 예측값(y_pred_rf)의 Accuracy를 구하고 rf_acc 변수에 저장하세요
 - 2개 모델의 Accuracy값을 출력하세요
- (12-2) 두 모델의 Accuracy 결과를 비교했을 때, 성능이 더 우수한 모델은 무엇인가요?

In []: # (12-1) 아래 코드의 오류를 정정하고 실행하세요

```
from sklearn.metrics import accuracy

y_pred_dt = dt.predict(X_valid)
y_pred_rf = rf.predict(X_valid)

dt_acc = accuracy(y_valid, y_pred_dt)
rf_acc = accuracy(y_valid, y_pred_rf)

print(dt_acc, rf_acc)
```

In []: # (12-2) 여기에 답안을 입력하세요(실행 불필요)

DecisionTree

In [15]: # [정답]

```
from sklearn.metrics import accuracy_score

y_pred_dt = dt.predict(X_valid)
y_pred_rf = rf.predict(X_valid)

dt_acc = accuracy_score(y_valid, y_pred_dt)
rf_acc = accuracy_score(y_valid, y_pred_rf)

print(dt_acc, rf_acc)
```

다음 문항을 풀기 전에 아래 코드를 실행하세요.

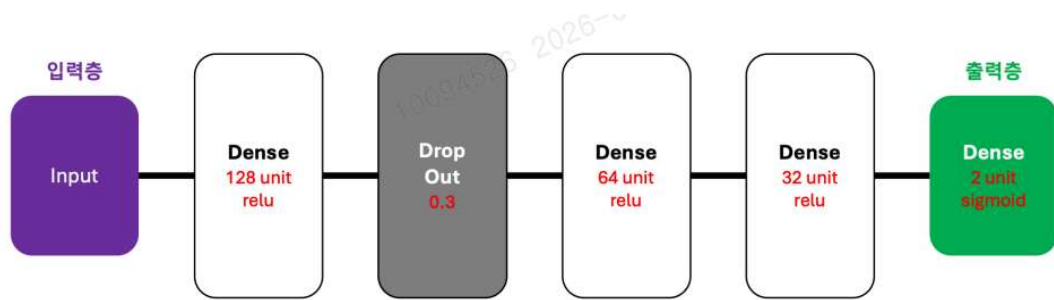
```
In [16]: import tensorflow as tf
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Dense, Activation, Dropout, BatchNormalizati
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.utils import to_categorical

tf.random.set_seed(1)
```

13. 와인품질등급(Grade)을 예측하는 딥러닝 모델을 만들려고 합니다.

아래 토폴로지 다이어그램과 가이드를 참고해서 모델링하고 학습을 수행하세요.

-
- tensorflow framework를 사용하고 하단의 토폴로지 그림과 동일한 딥러닝 모델을 구현하세요
 - 히든 레이어의 activation 함수는 'relu'를, 마지막 아웃풋 레이어의 activation 함수는 'sigmoid'를 사용하세요
 - EarlyStopping 함수를 사용해서 14번의 epoch 동안 모니터링 지표(val_loss)가 향상되지 않을 경우 훈련을 중지하도록 설정하고 estop 변수에 저장하세요
 - optimizer는 adam, metrics는 accuracy, loss는 binary_crossentropy로 지정해서 모델을 컴파일하세요
 - 다음 조건에 따라 모델을 학습하고 학습정보는 history 변수에 저장하세요
 - batch_size : 128
 - epoch : 50
 - 안내된 내용 외 별도의 파라미터를 입력하지 마세요
 - 아래 코드 셀(cell)에 있는 코드의 빈칸을 채우고 실행하세요
 - 빈칸에 들어갈 내용은 각 답안 셀(cell)에 입력하세요 (#13-1, #13-2)
-



```

In [ ]: # (코드 셀) 코드의 빈칸을 채우고 실행하세요

from sklearn.preprocessing import LabelEncoder

model = Sequential([
    <#13-1>
])

estop = EarlyStopping(monitor='val_loss', <#13-2> =14, restore_best_weights=True)
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

le = LabelEncoder()
y_train = le.fit_transform(y_train)
y_valid = le.transform(y_valid)

y_train = to_categorical(y_train)
y_valid = to_categorical(y_valid)

history = model.fit(X_train, y_train,
                    epochs=50,
                    batch_size=128,
                    validation_data=(X_valid, y_valid),
                    callbacks=[estop])
  
```

```

In [ ]: # (13-1) 여기에 답안을 입력하세요(실행 불필요)

Dense(128, activation='relu', input_dim=X_train.shape[1]),
BatchNormalization(),
Dropout(0.2),
Dense(64, activation='relu'),
BatchNormalization(),
Dense(32, activation='relu'),
BatchNormalization(),
Dense(2, activation='sigmoid')
  
```

```

In [ ]: # (13-2) 여기에 답안을 입력하세요(실행 불필요)

patience
  
```

```

In [17]: # [정답]

from sklearn.preprocessing import LabelEncoder

model = Sequential([
    Dense(128, activation='relu', input_dim=X_train.shape[1]),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
  
```

```
        Dense(2, activation='sigmoid')
    ])

    estop = EarlyStopping(monitor='val_loss', patience=14, restore_best_weights=True)
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

    le = LabelEncoder()
    y_train = le.fit_transform(y_train)
    y_valid = le.transform(y_valid)

    y_train = to_categorical(y_train)
    y_valid = to_categorical(y_valid)

    history = model.fit(X_train, y_train,
                        epochs=50,
                        batch_size=128,
                        validation_data=(X_valid, y_valid),
                        callbacks=[estop])
```

Epoch 1/50
438/438  3s 4ms/step - accuracy: 0.7824 - loss: 0.4609 - val_
accuracy: 0.8085 - val_loss: 0.4164

Epoch 2/50
438/438  2s 4ms/step - accuracy: 0.8037 - loss: 0.4235 - val_
accuracy: 0.8096 - val_loss: 0.4117

Epoch 3/50
438/438  2s 5ms/step - accuracy: 0.8063 - loss: 0.4176 - val_
accuracy: 0.8117 - val_loss: 0.4081

Epoch 4/50
438/438  2s 5ms/step - accuracy: 0.8080 - loss: 0.4136 - val_
accuracy: 0.8122 - val_loss: 0.4076

Epoch 5/50
438/438  4s 9ms/step - accuracy: 0.8088 - loss: 0.4096 - val_
accuracy: 0.8122 - val_loss: 0.4043

Epoch 6/50
438/438  2s 5ms/step - accuracy: 0.8105 - loss: 0.4085 - val_
accuracy: 0.8125 - val_loss: 0.4041

Epoch 7/50
438/438  2s 5ms/step - accuracy: 0.8113 - loss: 0.4071 - val_
accuracy: 0.8131 - val_loss: 0.4044

Epoch 8/50
438/438  2s 4ms/step - accuracy: 0.8112 - loss: 0.4063 - val_
accuracy: 0.8125 - val_loss: 0.4038

Epoch 9/50
438/438  2s 5ms/step - accuracy: 0.8114 - loss: 0.4045 - val_
accuracy: 0.8127 - val_loss: 0.4050

Epoch 10/50
438/438  3s 6ms/step - accuracy: 0.8120 - loss: 0.4034 - val_
accuracy: 0.8135 - val_loss: 0.4036

Epoch 11/50
438/438  2s 5ms/step - accuracy: 0.8123 - loss: 0.4029 - val_
accuracy: 0.8132 - val_loss: 0.4033

Epoch 12/50
438/438  2s 4ms/step - accuracy: 0.8137 - loss: 0.4019 - val_
accuracy: 0.8129 - val_loss: 0.4049

Epoch 13/50
438/438  2s 4ms/step - accuracy: 0.8132 - loss: 0.4017 - val_
accuracy: 0.8134 - val_loss: 0.4032

Epoch 14/50
438/438  2s 4ms/step - accuracy: 0.8129 - loss: 0.4012 - val_
accuracy: 0.8142 - val_loss: 0.4028

Epoch 15/50
438/438  2s 4ms/step - accuracy: 0.8134 - loss: 0.4008 - val_
accuracy: 0.8143 - val_loss: 0.4044

Epoch 16/50
438/438  3s 4ms/step - accuracy: 0.8144 - loss: 0.3992 - val_
accuracy: 0.8128 - val_loss: 0.4057

Epoch 17/50
438/438  2s 6ms/step - accuracy: 0.8144 - loss: 0.3990 - val_
accuracy: 0.8139 - val_loss: 0.4030

Epoch 18/50
438/438  3s 6ms/step - accuracy: 0.8140 - loss: 0.3997 - val_
accuracy: 0.8130 - val_loss: 0.4045

Epoch 19/50
438/438  2s 5ms/step - accuracy: 0.8140 - loss: 0.3994 - val_
accuracy: 0.8134 - val_loss: 0.4037

Epoch 20/50
438/438  2s 4ms/step - accuracy: 0.8156 - loss: 0.3980 - val_
accuracy: 0.8142 - val_loss: 0.4032

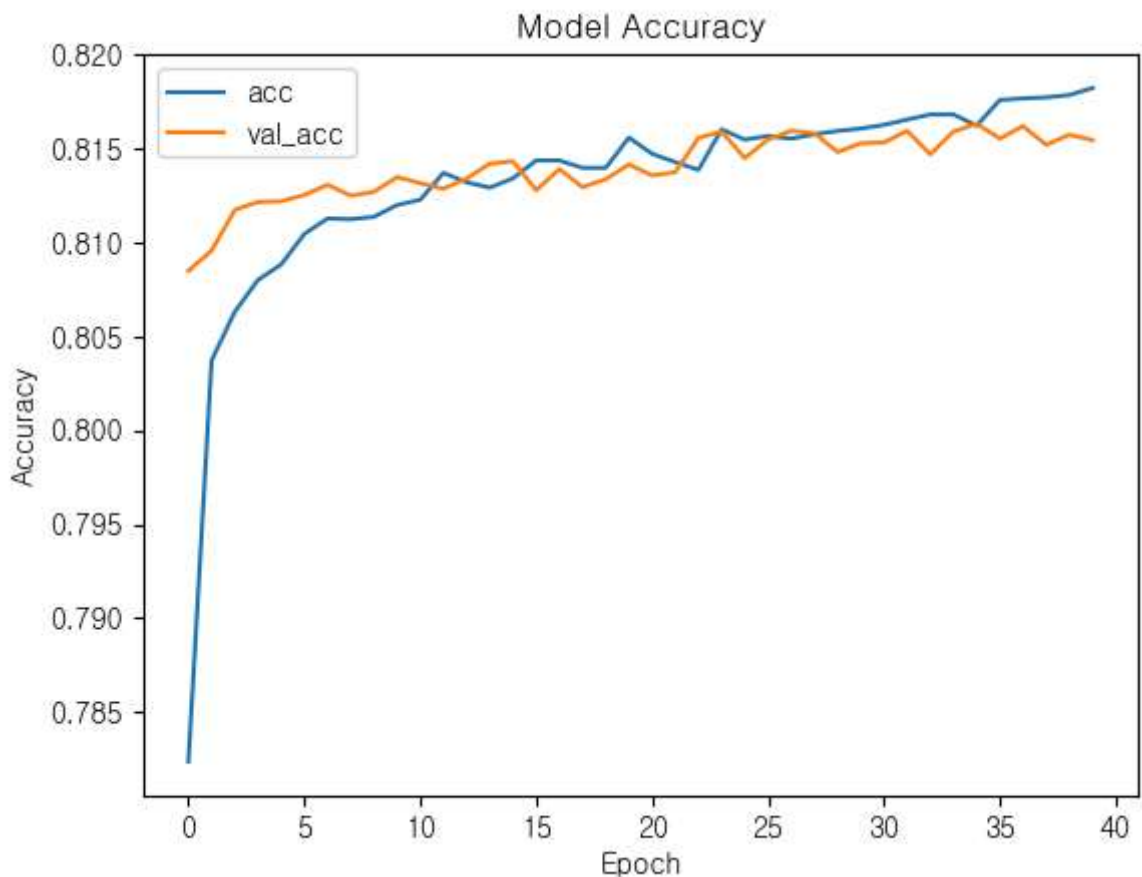
Epoch 21/50
438/438  2s 5ms/step - accuracy: 0.8147 - loss: 0.3973 - val_
accuracy: 0.8136 - val_loss: 0.4026
Epoch 22/50
438/438  3s 7ms/step - accuracy: 0.8143 - loss: 0.3972 - val_
accuracy: 0.8137 - val_loss: 0.4037
Epoch 23/50
438/438  3s 8ms/step - accuracy: 0.8139 - loss: 0.3970 - val_
accuracy: 0.8156 - val_loss: 0.4021
Epoch 24/50
438/438  2s 5ms/step - accuracy: 0.8160 - loss: 0.3972 - val_
accuracy: 0.8159 - val_loss: 0.4014
Epoch 25/50
438/438  3s 6ms/step - accuracy: 0.8155 - loss: 0.3968 - val_
accuracy: 0.8145 - val_loss: 0.4012
Epoch 26/50
438/438  4s 4ms/step - accuracy: 0.8157 - loss: 0.3965 - val_
accuracy: 0.8155 - val_loss: 0.4005
Epoch 27/50
438/438  2s 5ms/step - accuracy: 0.8155 - loss: 0.3956 - val_
accuracy: 0.8160 - val_loss: 0.4013
Epoch 28/50
438/438  2s 5ms/step - accuracy: 0.8158 - loss: 0.3959 - val_
accuracy: 0.8158 - val_loss: 0.4009
Epoch 29/50
438/438  2s 4ms/step - accuracy: 0.8159 - loss: 0.3954 - val_
accuracy: 0.8148 - val_loss: 0.4009
Epoch 30/50
438/438  2s 5ms/step - accuracy: 0.8161 - loss: 0.3953 - val_
accuracy: 0.8153 - val_loss: 0.4015
Epoch 31/50
438/438  2s 4ms/step - accuracy: 0.8163 - loss: 0.3943 - val_
accuracy: 0.8153 - val_loss: 0.4014
Epoch 32/50
438/438  3s 6ms/step - accuracy: 0.8166 - loss: 0.3941 - val_
accuracy: 0.8160 - val_loss: 0.4006
Epoch 33/50
438/438  2s 5ms/step - accuracy: 0.8168 - loss: 0.3946 - val_
accuracy: 0.8147 - val_loss: 0.4027
Epoch 34/50
438/438  2s 5ms/step - accuracy: 0.8168 - loss: 0.3935 - val_
accuracy: 0.8159 - val_loss: 0.4020
Epoch 35/50
438/438  2s 5ms/step - accuracy: 0.8162 - loss: 0.3934 - val_
accuracy: 0.8163 - val_loss: 0.4013
Epoch 36/50
438/438  3s 5ms/step - accuracy: 0.8176 - loss: 0.3939 - val_
accuracy: 0.8155 - val_loss: 0.4013
Epoch 37/50
438/438  3s 5ms/step - accuracy: 0.8177 - loss: 0.3931 - val_
accuracy: 0.8162 - val_loss: 0.4020
Epoch 38/50
438/438  3s 5ms/step - accuracy: 0.8177 - loss: 0.3929 - val_
accuracy: 0.8152 - val_loss: 0.4028
Epoch 39/50
438/438  3s 5ms/step - accuracy: 0.8179 - loss: 0.3925 - val_
accuracy: 0.8157 - val_loss: 0.4017
Epoch 40/50
438/438  2s 5ms/step - accuracy: 0.8182 - loss: 0.3926 - val_
accuracy: 0.8155 - val_loss: 0.4024

14. 앞서 만든 딥러닝 모델의 일반화 성능(Generalization Performance)을 평가하려고 합니다. matplotlib 라이브러리 활용해서, 학습 Accuracy와 검증 Accuracy의 변화를 그래프로 시각화하세요.

- 아래 가이드에 따라 Accuracy 변화를 시각화하는 코드를 완성하세요
 - 1개의 그래프에 학습 Accuracy와 검증 Accuracy 2가지를 모두 표시하세요
 - 위 2가지 각각의 범례를 'acc', 'val_acc'로 표시하세요
 - 그래프의 타이틀은 'Model Accuracy'로 표시하세요
 - X축에는 'Epochs'라고 표시하고 Y축에는 'Accuracy'라고 표시하세요

In [18]: # (14) 여기에 답안코드를 작성하고 실행하세요

```
plt.plot(history.history["accuracy"])
plt.plot(history.history["val_accuracy"])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(['acc', 'val_acc'])
plt.show()
```



모든 문항이 완료되었습니다. 수고하셨습니다.

'최종제출 및 종료'를 클릭하셔서 답안을 제출해 주시기 바랍니다.